

Проект “Tower Defense”

Постановка задачи

В рамках проектной работы необходимо разработать программный проект в жанре классического **Tower Defense**.

Проект должен демонстрировать применение методов объектно-ориентированного анализа, объектно-ориентированного проектирования и объектно-ориентированного программирования.

Разрабатываемая система должна представлять собой игру, в которой по игровому полю движутся волны противников, а игрок размещает оборонительные башни, чтобы не допустить прорыва врагов к базе.

Главная цель работы заключается не только в создании работающего приложения, но и в построении корректной объектной модели предметной области, отражающей структуру и поведение игровых объектов.

Цель работы

Цель проектной работы — освоить полный цикл разработки объектно-ориентированной программной системы на примере игры жанра Tower Defense, включая:

- анализ предметной области;
- выделение сущностей, их атрибутов и поведения;
- проектирование взаимодействия объектов;
- реализацию программной системы на основе объектно-ориентированного подхода;
- тестирование ключевой логики проекта.

Требования к проекту

Программная система должна поддерживать базовую игровую механику жанра Tower Defense.

В игре должно быть реализовано игровое поле, по которому противники движутся от точки появления к точке назначения по заданному маршруту или маршрутам. Игрок должен иметь возможность размещать оборонительные башни в допустимых зонах игрового поля. Размещенные башни должны автоматически обнаруживать противников в радиусе действия и атаковать их в соответствии со своими характеристиками.

Противники должны перемещаться по маршруту, обладать запасом здоровья, скоростью передвижения и другими характеристиками. При уничтожении противника игрок получает игровые ресурсы. Если противник достигает конечной точки маршрута, база игрока получает урон или теряет единицы прочности. Игра продолжается до тех пор, пока либо все волны не будут успешно отражены, либо прочность базы не станет равной нулю.

Проект должен включать как минимум:

- не менее **трех типов башен**;
- не менее **трех типов противников**;
- механизм **волн врагов**;
- систему **игровых ресурсов**;
- механизм **нанесения урона**;
- условие **победы и поражения**.

Требования к объектно-ориентированному анализу

Перед началом программной реализации нужно выполнить анализ предметной области и определить ключевые сущности системы.

В рамках анализа должны быть выделены:

- основные игровые объекты;
- их характеристики;
- их действия;
- связи между объектами;

- ограничения и правила предметной области.

Результатом анализа должны стать ответы на следующие вопросы:

- какие сущности существуют в игре;
- какие из них являются основными;
- какими свойствами обладает каждая сущность;
- какое поведение характерно для каждой сущности;
- какие объекты взаимодействуют друг с другом;
- какие события изменяют состояние системы.

Требования к объектно-ориентированному проектированию

На этапе проектирования необходимо построить объектную модель системы.

При проектировании необходимо показать использование ключевых принципов ООП:

- инкапсуляции;
- наследования и/или интерфейсов;
- полиморфизма.

Объектная модель не должна сводиться к одному “главному” классу, внутри которого сосредоточена почти вся логика. Логика системы должна быть распределена между объектами в соответствии с их ответственностями.

В составе проектной документации нужно подготовить следующие диаграммы:

- диаграмму вариантов использования (Use-Case Diagram);
- диаграмму классов (Class Diagram);
- от 5 до 7 диаграмм последовательностей (Sequence Diagram), отражающих основные сценарии работы системы.

Функциональные требования

Игра должна обеспечивать следующий минимальный функционал.

После запуска игры пользователь должен иметь возможность начать игровую сессию. На игровом поле должна отображаться информация о состоянии базы, количестве доступных ресурсов, текущей волне и состоянии размещённых башен.

Пользователь должен иметь возможность:

- размещать башни на доступных позициях;
- запускать следующую волну противников;
- наблюдать за ходом боя;
- получать информацию о победе или поражении.

Каждая башня должна обладать собственными характеристиками, например:

- стоимостью;
- дальностью атаки;
- скоростью атаки;
- величиной урона.

Каждый противник должен обладать собственными характеристиками, например:

- запасом здоровья;
- скоростью движения;
- наградой за уничтожение;
- типом поведения или дополнительной особенностью.

В системе должны корректно обрабатываться следующие события:

- размещение башни;
- атака башни по противнику;
- получение противником урона;
- уничтожение противника;
- достижение противником базы;
- завершение волны;

- завершение игры.

Нефункциональные требования

Программа должна иметь понятную структуру и быть разбита на логические модули.

Имена классов, методов, переменных и других элементов программы должны быть осмысленными и отражать их назначение.

Код должен быть оформлен в соответствии с принятыми правилами именования и стилем выбранного языка программирования.

Проект должен быть пригоден для расширения: добавление нового типа башни или нового типа противника не должно требовать полной переработки архитектуры.

Проект должен иметь графический пользовательский интерфейс. Реализация только консольного или псевдографического интерфейса не допускается.

Логика предметной области должна быть отделена от графического интерфейса. Архитектура проекта должна обеспечивать возможность изменения или расширения интерфейса без полной переработки доменной модели игры.

Требования к реализации

В программной реализации должны быть представлены:

- пользовательские классы;
- методы экземпляров и, при необходимости, методы классов;
- использование коллекций объектов;
- обработка исключительных ситуаций;
- работа с файлами или другим механизмом сохранения данных, если это предусмотрено проектом.

Желательно предусмотреть:

- сохранение и загрузку состояния игры;
- конфигурирование параметров башен и врагов из внешних данных;

- разделение логики игры и пользовательского интерфейса.

Особенно желательно, чтобы явно отделяли:

- доменную модель игры;
- управляющую игровую логику;
- визуализацию или интерфейс взаимодействия.

Требования к тестированию

Студент должен продемонстрировать проверку корректности ключевой логики проекта.

Тестирование может быть выполнено с помощью модульных тестов, интеграционных тестов либо набора воспроизводимых сценариев.

Обязательной проверке подлежат как минимум:

- корректность списания ресурсов при размещении башни;
- корректность запрета на размещение башни при недостатке ресурсов;
- корректность движения противника по маршруту;
- корректность определения противников, находящихся в зоне действия башни;
- корректность нанесения урона противнику;
- корректность удаления уничтоженного противника;
- корректность начисления награды за уничтожение противника;
- корректность уменьшения прочности базы при достижении её противником;
- корректность определения состояний победы и поражения.

Технический стек

Проект должен быть реализован на языке программирования Python.

Для разработки графического пользовательского интерфейса необходимо использовать одну из следующих библиотек:

- PyQt5 / PyQt6;
- PySide2 / PySide6;
- tkinter;
- pygame.

Сборка

- Сборка проекта с использованием setuptools.
 - Упаковка в исполняемый файл с использованием PyInstaller.
-

Критерии оценивания

Оценивание проекта осуществляется по следующим направлениям:

- полнота и корректность анализа предметной области;
- качество диаграмм и обоснованность проектных решений;
- качество программной реализации;
- корректность работы системы и полнота реализации функциональных требований;
- качество тестирования ключевой логики;
- качество графического пользовательского интерфейса.

Важно! В проекте должно быть не менее 300 коммитов!